

Programmazione Web e Mobile con Laboratorio

Lezione 15 - Connessione al DB
(mysql2, pool, transactions)

DB e Node.js

- Per connettersi a un db in Node.js si usa una libreria specifica per il db
- Il processo è simile per la maggior parte dei casi:
 1. Installare la libreria adatta (mysql2, pg, mongoose, ...)
 2. Configurare la connessione fornendo le credenziali (host, porta, utente, password, nome db)
 3. Aprire la connessione tramite un metodo fornito dalla libreria
 4. Eseguire le query sul database
 5. Gestire gli errori e, se necessario, chiudere la connessione

Alcuni DB comuni

	MySQL	PostgreSQL	MongoDB
Tipo	Relazionale (SQL)	Relazionale avanzato (SQL)	NoSQL documentale (JSON-like)
Schema	Rigidamente definito	Rigidamente definito ma estendibile	Flessibile (schema-less)
Supporto JSON	Parziale	Nativo e potente	Nativo (BSON)
Transazioni	ACID (Atomicità, Coerenza, Isolamento, Durabilità)	ACID completo	Limitate (transazioni multi-doc recenti)
Scalabilità	Verticale	Orizzontale (sharding)	Orizzontale (sharding)
Prestazioni	Buone	Ottime anche su query complesse	Ottime su grandi volumi e scritture
Conformità SQL	Media	Elevata	N/D (usa query JSON-like)
Caso d'uso tipico	Web app, CMS, ecommerce	App complesse, analytics, Business Intelligence	App distribuite, IoT, prototipi rapidi

Operazioni CRUD

- Sono le quattro operazioni di base che si possono eseguire su qualsiasi data store
 - **Create**: inserimento di nuovi dati nel sistema
 - **Read**: recupero dei dati esistenti per visualizzarli o analizzarli
 - **Update**: modifica di dati esistenti per correggerli o aggiornarli
 - **Delete**: rimozione di dati che non sono più necessari
- Applicazioni comuni: database, API per interagire con le risorse

MySQL per Ubuntu

- Installare MySQL: `sudo apt-get install mysql-server`
- Verificare la versione: `mysql --version`
- Avviare il servizio: `sudo systemctl start mysql`
- Verificare lo stato: `sudo systemctl status mysql`
- Sicurezza: `sudo mysql_secure_installation`
- Accedere alla shell MySQL: `sudo mysql -u root -p`

Creare DB e tabelle

1. CREATE DATABASE mio_db;

2. SHOW DATABASES;

3. USE mio_db;

4. CREATE TABLE utenti (

 id INT AUTO_INCREMENT PRIMARY KEY,

 nome VARCHAR(100) NOT NULL,

 email VARCHAR(100) NOT NULL

);

5. SHOW TABLES; / DESCRIBE utenti;

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_inc.
nome	varchar(100)	NO		NULL	
email	varchar(100)	NO		NULL	

Nuovo utente e permessi

1. `CREATE USER 'user1'@'localhost' IDENTIFIED BY 'pass1';`
2. `SELECT user, host FROM mysql.user;`
3. `GRANT ALL PRIVILEGES ON mio_db.* TO 'user1'@'localhost';`
4. `FLUSH PRIVILEGES;`
5. `SHOW GRANTS FOR 'user1'@'localhost';`

mysql2

- Libreria Node.js che fornisce un'interfaccia a MySQL
- Caratteristiche:
 - Compatibile con MySQL 5.5+ e MariaDB
 - Interfaccia asincrona: promises, async/await e callback
 - Supporta gestione pool e connessioni persistenti
- Installazione: `npm install mysql2`

Connessione al DB

- `mysql.createConnection()` crea una connessione al db
- Il metodo `.connect()` apre la connessione al db

```
const connection = mysql.createConnection({
  host: "localhost",
  user: "user1",
  password: "pass1",
  database: "mio_db"
});

connection.connect((err) => {
  if (err) {
    console.error("Errore nella connessione al database: " + err.stack);
    return;
  }
  console.log("Connesso al database con ID " + connection.threadId);
});
```

Esecuzione della query

- `connection.execute()` esegue una query parametrizzata
- `query` è una stringa SQL con placeholder (?)
- `[nome, email]` è l'array dei valori da usare nei placeholder

```
const query = "INSERT INTO utenti (nome, email) VALUES (?, ?)";

connection.execute(query, [nome, email], (err, results) => {
  if (err) { // Internal Server Error
    console.error("Errore: " + err.stack);
    return res.status(500).json({ success: false, message: "Errore!" });
  }
  return res.json({ success: true, message: "Successo!" });
});
```

Esercizio 1

- Partendo dall'esempio precedente
 1. Nel server, implementare una nuova route GET che interroga il db e restituisce la lista degli utenti
 2. Nel client, utilizzare fetch per richiedere la lista utenti dal server e visualizzala dinamicamente nella pagina

Aggiungi nuovo utente

Nome:

Email:

Aggiungi utente

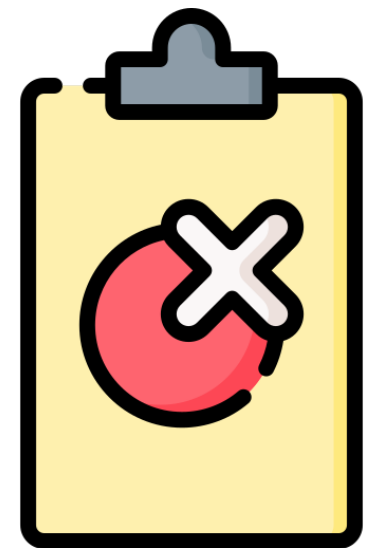
Leggi lista utenti

Lista utenti

- Aldo Baglio - aldoba@mail.it
- Giovanni Storti - giosto@mail.it
- Gicomo Poretti - giapor@email.it

Connessione globale

- `mysql.createConnection()` crea una singola connessione persistente tra Node.js e MySQL
- Rimane aperta finché non viene chiusa esplicitamente con `connection.end()`
- Non va bene per servire più richieste HTTP:
 - può andare in timeout e chiudersi da sola
 - può causare problemi di concorrenza



Connessione isolata

- `createConnection()` e `.execute()` dentro ogni route GET/POST
- Pro: sicura per la concorrenza: ogni richiesta ha la sua connessione
- Con: costi elevati per gestire apertura e chiusura della connessione ad ogni richiesta



Connessione condivisa

- `createConnection()` globale e solo `.execute()` dentro ogni route GET/POST
- Vantaggi: solo apparenti
- Svantaggi: gli stessi problemi della connessione globale



Connection pool

- Insieme di connessioni mantenute attive e riutilizzate per le richieste concorrenti
- Il pool gestisce:
 - richieste concorrenti
 - riuso efficiente delle connessioni
 - rilascio automatico della connessione al termine della query



Esempio di pool

```
const pool = mysql.createPool({
  host: "localhost",
  user: "user1",
  password: "pass1",
  database: "mio_db",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

app.get("/users", async (req, res) => {
  const query = "SELECT * FROM utenti";
  const connection = await pool.promise().getConnection();
  try {
    const [righe, colonne] = await connection.execute(query);
    res.json(righe);
  }
  catch (err) {
    console.error("Errore durante l'esecuzione della query:", err);
    res.status(500).json({success: false, message: "Errore" });
  }
  finally {
    connection.release();
  }
});
```

Configurazione del pool

- `createPool()` crea un pool di connessioni al db
- `waitForConnections` decide se le richieste devono aspettare una connessione libera quando il pool è pieno
- `connectionLimit` imposta un massimo di connessioni simultanee nel pool
- `queueLimit` imposta un numero massimo di richieste in attesa
- `.promise()` serve per usare `await` con `.execute()`
- `.release()` rilascia la connessione

Transactions

- Una transazione è un'unità di lavoro atomica che raggruppa una o più operazioni di db in un'unica operazione logica
- Le transazioni garantiscono che tutte le operazioni siano eseguite con successo o nessuna di esse venga applicata in caso di errore
- Esempio pratico: trasferimento di denaro tra due conti bancari
 1. Addebito sul primo conto
 2. Accredito sul secondo conto
- Se una di queste operazioni fallisce, la transazione garantisce che nessuna delle operazioni venga applicata

Esempio di transaction

```
app.post("/registration", async (req, res) => {
  const { nome, email, evento_id, posto_id, importo } = req.body;
  const connection = await pool.promise().getConnection();
  try {
    await connection.beginTransaction();

    // 1. Inserisco l'utente come partecipante
    const insertPart = "INSERT INTO partecipanti (nome, evento_id) VALUES (?, ?)";
    await connection.execute(insertPart, [nome, evento_id]);

    // 2. Segno il posto come prenotato
    const insertPrenot = "INSERT INTO posti (evento_id, posto_id, nome) VALUES (?, ?, ?)";
    await connection.execute(insertPrenot, [evento_id, posto_id, nome]);

    // 3. Registro il pagamento
    const insertIncasso = "INSERT INTO incassi (evento_id, nome, importo) VALUES (?, ?, ?)";
    await connection.execute(insertIncasso, [evento_id, nome, importo]);
    await connection.commit();
    console.log("Transazione completata con successo");
    return res.json({ success: true, message: "Successo!" });
  }
  catch (err) {
    await connection.rollback();
    console.error("Errore nella transazione, rollback:", err);
    return res.status(500).json({ success: false, message: "Errore!" });
  }
  finally {
    connection.release();
  }
});
```

Metodi per transaction

- `await pool.promise().getConnection()` ottiene una connessione dal pool
- `await connection.beginTransaction()` avvia una transazione in modo asincrono
- `await connection.execute()` esegue una singola query in modo asincrono
- `await connection.commit()` rende permanenti le modifiche una volta che tutte le operazioni della transazione sono state completate con successo
- `await connection.rollback()` annulla tutte le modifiche effettuate se si verifica un errore durante la transazione
- `connection.release()` rilascia la connessione al termine della transazione e la restituisce al pool

Esercizio 2

id	nome	categoria	prezzo	disponibile	data_inserimento
1	Laptop X	Elettronica	999.99	1	2025-05-14 12:59:09
2	Mouse Pro	Accessori	49.90	1	2025-05-14 12:59:09
3	Sedia Ergonomica	Arredamento	199.00	0	2025-05-14 12:59:09
4	Tastiera Meccanica	Accessori	129.50	1	2025-05-14 12:59:09
5	Monitor 4K	Elettronica	349.99	1	2025-05-14 12:59:09

- Creare una route GET /prodotti che riceve i parametri categoria, disponibile e maxPrezzo tramite query string
- Eseguire una SELECT che usa i parametri ricevuti
- Ritornare un oggetto JSON che contiene i prodotti trovati
- Esempio di richiesta:
/prodotti?categoria=accessori&disponibile=1&maxPrezzo=500